Computers & Industrial Engineering 199 (2025) 110710



Contents lists available at ScienceDirect

Computers & Industrial Engineering

journal homepage: www.elsevier.com/locate/caie





Integrated problem of car sequencing and vehicle routing on an automotive mixed-model assembly line

Jian Chen, Hong Zhou, Qiang Xue, Naiming Xie, Yao-Wen Sang *

College of Economics and Management, Nanjing University of Aeronautics and Astronautics, Nanjing 211106, People's Republic of China

ARTICLE INFO

Keywords:
Multi-trip vehicle routing
Mixed-model assembly line
Sequencing
Adaptive large neighborhood search
Sync-part supply

ABSTRACT

We address an integrated problem of car sequencing on an automotive mixed-model assembly line and multi-trip vehicle routing for external sync-part supply. The sync-parts, usually characterized by large size and highly customized, are directly delivered from suppliers to the assembly line without intermediate inventory. The goal is to find a joint schedule of car sequencing and vehicle routing so as to minimize the overall makespan. To tackle this intricate problem, we develop a mixed-integer linear programming model by employing the constraint linearization technique. A new meta-heuristic named MS-ALNS-VND is proposed combining multiple start strategy, adaptive large neighborhood search (ALNS) and variable neighborhood descent (VND). Numerical experimental results show that MS-ALNS-VND can solve small-sized instances to optimality in much less running time compared to Gurobi. For medium- and large-sized instances, MS-ALNS-VND outperforms ALNS and VND in both best or average objective values. A case study from an electric automobile manufacturer in China demonstrates the applicability and efficiency of our algorithm.

1. Introduction

Automobile manufacturers, especially those producing electric automobiles, are facing growing demands for product customization and small-batch production. Various job variants or models with different specifications are processed on a mixed-model assembly line, resulting in a diverse range of part demands (Schmid et al., 2021). Manufacturers proactively maintain a certain level of inventory in preparation for meeting the demands of small parts on the assembly line (Blanchard, 1983). However, for certain parts characterized by large size and high degree of customization, such as seats and doors, stockpiling them in warehouses would result in substantial inventory usage. Hence, a strategy through synchronized part supply and production is proposed by manufacturers to meet the demand for these customized parts. This aims to reduce inventory levels and ensure uninterrupted production. We designate those parts that undergo processing at sync-workstations and exhibit these specific attributes as "sync-parts".

During the synchronized supply of sync-parts, vehicles depart from the manufacturer, travel to suppliers to load the sync-parts, and then return to the manufacturer. Subsequently, the sync-parts are directly delivered to sync-workstations on the mixed-model assembly line. Sync-workstations are located sequentially on the mixed-model assembly line. A job sequentially arrives at these sync-workstations. Usually, the flow time of a job between the first and last sync-workstations needs less than an hour. In order for uninterrupted production, sync-parts are delivered to their sync-workstations on their due dates. The vehicles maintain a continuous back-and-forth journey between the manufacturer and suppliers, ensuring a seamless supply of parts. At this point, transportation and production proceed in a synchronized manner. Adopting the synchronized supply of sync-parts mitigates inventory expenses because of the direct delivery of the sync-parts to sync-workstations. The basic setting is depicted in Fig. 1.

The milk-run logistics is extensively utilized across the automobile manufacturers for the efficient delivery of a diverse array of automotive parts (Facchini et al., 2024; Mao et al., 2020; Ranjbaran et al., 2020). The milk-run logistics model is characterized by a series of sequential stops at different suppliers, allowing a single vehicle to collect various parts and then deliver them to the manufacturer in a consolidated manner (Nemoto et al., 2010). In the automotive industry, aside from milk-run logistics, common delivery strategies include point-to-point and cross-docking. However, these alternatives can be costly for delivering sync-parts. Point-to-point, involving direct shipments from a single supplier to a manufacturer, may lead to high costs due to the extensive vehicle usage required for sync-part logistics with multiple suppliers. While cross-docking, which aggregates parts from various suppliers before unified transport, enhances efficiency, it can incur

* Corresponding author.
E-mail addresses: jchen@nuaa.edu.cn (J. Chen), hongzhou_ie@163.com (H. Zhou), xueqiang@nuaa.edu.cn (Q. Xue), xienaiming@nuaa.edu.cn (N. Xie), ywsang@nuaa.edu.cn (Y.-W. Sang).

https://doi.org/10.1016/j.cie.2024.110710
Received 11 June 2024; Received in revised form 30 September 2024; Accepted 5 November 2024
Available online 19 November 2024
0360-8352/© 2024 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

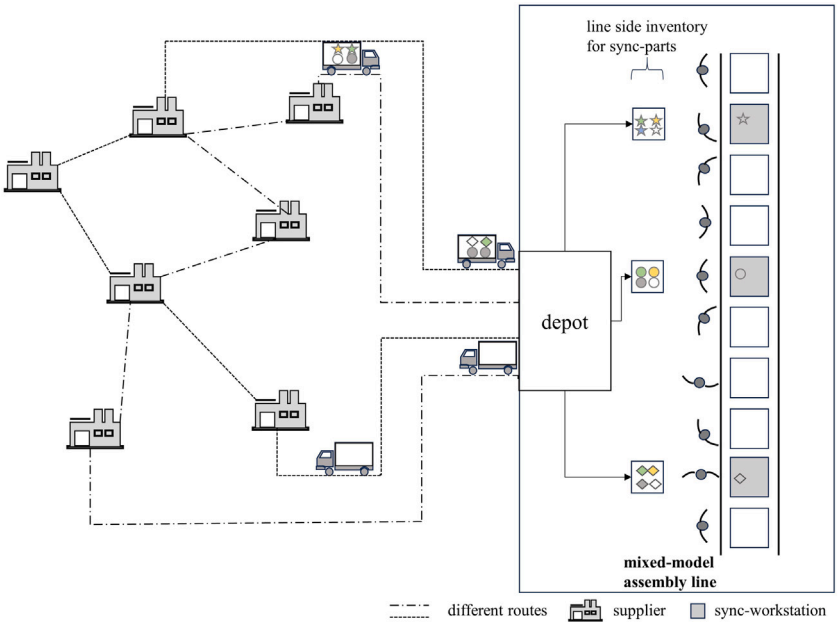


Fig. 1. A schematic chart of vehicle routing for sync-parts supply.

significant storage costs due to the accumulation of parts. In contrast, milk-run logistics effectively balance transportation and storage costs for sync-parts. To capture the benefits of milk-run, we incorporate it into the transportation of sync-parts, where vehicles visit multiple suppliers to consolidate parts for transportation, thereby optimizing the logistics process and reducing overall costs. As depicted in Fig. 1, the transportation of sync-parts requires multiple trips of vehicles to meet the due dates of sync-parts, emphasizing the importance of the coordination between logistics and production.

It is imperative that the sync-parts are delivered to the manufacturer before their individual due dates so that the mixed-model assembly line will not stop because of waiting sync-parts. On the mixed-model assembly line, the car sequencing determines the exact due dates of sync-parts. Different car sequencing leads to distinct due dates of sync-parts, affecting decisions of varying transportation schemes. Integrating production and transportation scheduling presents significant opportunities for optimization (Ehm & Freitag, 2016). This inspires us to concurrently address the integrated car sequencing and vehicle routing problem (ICSVRP). Part delivery is carried out by a fleet of homogeneous vehicles. Considering the capacity of vehicles and synchronized supply, the vehicles need to make multiple trips between the manufacturer and suppliers. Therefore, ICSVRP involves optimizing the decision of multi-trip vehicle routing problem (MTVRP). In the ICSVRP problem, part delivery is pulled by production, which differs from a stream of research that considers integrated production and outbound distribution scheduling (IPODS) (Chen, 2010).

In order to address the intricate problem, a non-linear mixed integer programming model is built and linearized to be a mixed integer linear programming model (MILP). This model can be solved by commercial solvers, such as Gurobi. But for medium- and large-sized instances, it needs considerable computing time. Inspired by the adaptive large neighborhood search and variable neighborhood descent framework (Pan et al., 2021), we propose a new meta-heuristic (called MS-ALNS-VND) incorporates multiple start (MS) strategy, adaptive large neighborhood search (ALNS) and variable neighborhood descent (VND).

In addition, a novel encoding operator is devised to transform the delivery-assembly pull problem into an assembly-delivery push problem. Using the encoding operator, the hard constraint imposed by the due dates of sync-parts can be easily addressed. Moreover, we design swap, destroy, and local search operators that are customized to the problem's features to boost the algorithm's performance. To further avoid getting trapped in local optima, we reinforce the perturbation process with a simulated annealing strategy. For small-sized instances, our algorithm is compared to the optimal solutions solved by Gurobi, while for medium- and large-sized instances, it is compared to ALNS-VND and ALNS.

The main contributions of this paper are threefold. Firstly, though multi-trip vehicle routing or car sequencing problems have been independently studied, limited research investigates integrated car sequencing and vehicle routing problems under sync-part delivery mode. Secondly, we propose a MILP model, which can be addressed by commercial solvers, such as Gurobi. Thirdly, we propose a new MS-ALNS-VND incorporating multiple start strategy, ALNS and VND, which can greatly balance the breadth and depth of the search space. Besides, we propose a novel encoding operator in which the hard constraint imposed by the due dates of sync-parts can be easily addressed.

The rest of this paper is structured as follows. Section 2 provides a literature review. Section 3 describes the proposed nonlinear mixed integer programming model and its linearization. In Section 4, we present the MS-ALNS-VND algorithm for the problem. In Section 5, we conduct numerical experiments and validate the effectiveness and performance of the proposed algorithm. In Section 6, we present a real-world case study. Section 7, we conclude this research.

2. Literature review

The literature review is structured as follows. We first review the literature on car sequencing on the mixed-model assembly line. Next, we investigate internal and external part supply in the automobile supply chain. Finally, we discuss the integrated production and outbound distribution scheduling problem, which is similar to our ICSVRP.

2.1. Sequencing problem

Sequencing problem on the mixed-model assembly lines has a long history, mainly involving various optimization approaches to support sequencing decision (Boysen et al., 2009). Tamura et al. (2011) considered the sequencing problem to minimize the stoppage time on the mixed-model assembly line. In this problem, if the assembly operation exceeds predetermined process times, the conveyor comes to

a halt. A branch-and-bound algorithm was developed to solve this problem. Bautista and Cano (2011) aimed to minimize the workload on mixed-model assembly lines with serial workstations. They proposed a bounded dynamic programming to solve this problem effectively. Fattahi and Salehi (2009) addressed the variable launching interval and sequencing problem, optimizing idle time and workloads by dynamically adjusting the launching interval. A simulated annealing algorithm was proposed to solve it.

Different from the classic sequencing problems, Özcan et al. (2010) combined the balancing and sequencing problem to maximize the line efficiency and smooth workloads, and they developed a simulated annealing to solve it. Defersha and Mohebalizadehgashti (2018) also considered the combined problem but optimized the line length, the number, and the cost of the workstations. Striving for a solution of superior quality, a multi-phased linear programming embedded genetic algorithm was proposed. Lopes et al. (2020) studied the combination of buffer allocation, balancing, and sequencing on asynchronous mixed-model assembly lines. Buffers are assigned to the line to enhance the system's productivity by storing incomplete products. Due to the complexity of the integration problem, an iterative decomposition procedure was developed. Tiacci (2024) further advanced the complex field and considered the objective of maximizing assembly line efficiency and minimizing configuration costs. Some car sequencing studies transfer these complex attributes to the simple rules (Golle et al., 2014; Parrello et al., 1986; Pontes et al., 2024; Sun, 2024; Wang et al., 2022). In addition to the automotive industry, sequencing problems are also across various industries such as home appliance manufacturing (Elyasi et al., 2024), reverse logistics (Tasoglu & Ilgin, 2024), warehouse picking (Cao et al., 2023), metal working (Calmels, 2022), and so on. However, these studies primarily focus on optimizing the job sequencing on the assembly line.

2.2. Part supply in automobile industry

Some research has studied internal part supply for mixed-model assembly lines. Boysen and Emde (2014) investigated the part supply problem with line-integrated supermarkets and mixed-model assembly lines, aiming to arrange the minimum number of workers to meet part demands in each production cycle using just-in-sequence (JIS) part packing. Similarly, Emde and Boysen (2012) considered JIT supply of parts stored in different supermarkets, with tow-trains passing through different stations to supply parts. The determination of routes was treated as a vehicle routing problem. Zhou and Zhu (2020) applied the milk-run logistics to internal part supply on mixed-model assembly lines, aiming to minimize the total line side storage in a planning horizon. Facchini et al. (2024) investigated the integration of milk-run logistics and Internet of Things (IoT), employing IoT for collecting sensor data from vehicles and implementing milk-run logistics to deliver the materials to different workstations of mixed-model assembly lines.

In addition, the external part supply has also been studied, and the milk-run logistics is a commonly employed method for it. Mao et al. (2020) considered a new inbound part milk-run logistics using progress-lane, which is a terminal buffer of milk-run, catering to the demands of assembling. The problem was formulated as a single trip vehicle routing problem. Sadjadi et al. (2009) proposed a new milk-run method for parts supply in the automobile industry: grouping suppliers and assigning one vehicle to transport parts for each group. The objective was to determine the cost-effective vehicle routing, where each vehicle may visit multiple groups but only a single trip. This method demonstrated significant advantages for high-value parts and short-distance suppliers. Novaes et al. (2015) examined milk-run logistics while also incorporating dynamic vehicle routing of a single trip, assigning tasks exceeding time limits to additional vehicles. The results showed that the dynamic formulation significantly enhanced service levels compared to the static. Ranjbaran et al. (2020) considered milk-run logistics with forward and reverse flow of pallets to address part supply in the actual automobile industry, revealing significant advantages of the milk-run compared to direct shipping strategy. However, most milk-run studies ultimately address the optimization of a single trip vehicle routing problem, and limited studies have considered the combination of car sequencing and external sync-part logistics.

Our problem shares a similar background with the aforementioned studies, aiming to transport parts from different suppliers to the manufacturer. However, their research gap can be easily identified. Given the frequent demand for parts on the mixed-model assembly lines and the small-batch transportation mode, our transportation part can be considered as a multi-trip vehicle routing problem, where each vehicle may visit different suppliers for different trips. The due dates of parts are determined by the sequencing on the mixed-model assembly line. The coordination between production and logistics makes the joint problem more complex.

2.3. Difference with IPODS

A stream of research similar to our ICSVRP is IPODS problem. ICSVRP shares some similarities with the IPODS, for example, coordination between production and transportation (Ullrich, 2013), the characteristics of the fleet (Dayarian & Desaulniers, 2019), and the optimization of vehicle routing (Kergosien et al., 2017). The IPODS problem was initially studied by Potts (1980), who considered the use of infinite homogeneous vehicles with single-trip vehicle routing problem. To solve the IPODS problem, Low et al. (2013) considered a single-machine problem with transportation conducted by a fleet of homogeneous vehicles, aiming to minimize the time for the last vehicle returning to the depot within the time windows. Similarly, considering a single-machine problem with time windows, Low et al. (2017) investigated heterogeneous vehicle routing with the objective of minimizing the total costs, including vehicle fixed costs, travel costs, and costs due to time window violations. They proposed a MILP model and an adaptive algorithm incorporating local search.

Belo-Filho et al. (2015) considered parallel machines for the production and distribution of perishable goods with time windows. They developed an ALNS algorithm to solve the integrated problem. Chang et al. (2013) considered a set of parallel machines and a fleet of capacitated homogeneous vehicles. The master problem was decomposed into a restricted master and several subproblems by column generation, and each subproblem, representing a single machine and single batch delivery problem, is solved by dynamic programming.

Our ICSVRP includes the MTVRP transportation, which is investigated in a few IPODS studies. Karaoğlan and Kesen (2017) considered single-machine production and capacity-constrained single-vehicle and multi-trip transportation, where the production and delivery sequences were the same. The objective was to minimize the time for the vehicle to return after the last trip. The authors used a branch-and-cut algorithm to solve the problem and compared it with a genetic algorithm. Ehm and Freitag (2016) considered a hybrid flow shop with vehicles of different capacities, where each job had a release time and a due date. To minimize a comprehensive cost, they proposed a MILP model and three heuristic algorithms: classical variable neighborhood search, a construction heuristic, and a hybrid algorithm combining the previous two. Martins et al. (2021) considered an open shop scheduling problem with transportation performed by a single capacitated vehicle. They used a new biased random key genetic algorithm with an iterated greedy local search procedure to solve the problem, and experimental results showed that the algorithm outperformed other tested algorithms. Nagano et al. (2022) used an iterated greedy algorithm to determine the sequence of orders in the permutation flow shop production and assign them to different vehicles in the distribution.

To summarize, our research has identified several gaps in the current literature. Firstly, while the sequencing and part logistics problems have been widely studied, their integration has received limited attention. Secondly, although both ICSVRP and IPODS address

the integration of production and transportation, ICSVRP focuses on delivery-to-production integration, in contrast to IPODS which follows a production-to-delivery integration approach. Thirdly, to address the ICSVRP, this paper proposes a new meta-heuristic algorithm (called MS-ALNS-VND), which incorporates multiple start strategy, ALNS, and VND.

3. Problem description and modeling

3.1. Problem description

At each planning horizon, an electric automobile manufacturer has given a number of jobs (cars) with different types (i.e., models) to produce on a paced mixed-model assembly line. The number and type of jobs are known in advance according to the master production schedule. For each job, there are several sync-parts that require synchronized (direct) delivery from suppliers to the sync-workstations. This is general because of the constraints of limited line-side buffer and large size of sync-parts.

There is a fleet of homogeneous vehicles for part delivery. In order to utilize the vehicles, they depart from the manufacturer and travel to the respective suppliers in the milk-run network. Note that a sync-workstation cannot work until its sync-parts are arrived. In order to maintain production continuity, these sync-parts are subjected to hard due dates based on the car sequencing. The transportation problem can be viewed as a MTVRP.

Therefore, the entire problem aims to address an integrated optimization of sequencing and MTVRP so that the makespan is minimized. The sequencing is to determine the car sequencing on the mixed-model assembly line. This certainly generates the individual due dates of sync-parts. The vehicle routing is to determine the assignment of parts to different trips, routing of each trip, and the arrival time of each visit considering various constraints. Different vehicle routing will certainly determine the arrival of syn-parts, affecting the start time of production at each sync-workstation. The entire problem is defined as ICSVRP in this paper. The ICSVRP can be mathematically described as follows.

The problem is represented on a complete undirected graph $G = \{V, E\}$. The vertex set $V = \{0, n+1\} \cup N$ consisting of $n+2$ vertices, where vertices 0 and $n+1$ represent a manufacturer (depot), and $N = \{1, 2, \dots, n\}$ represents the set of suppliers. $E = \{(i, j), i, j \in V\}$ represents a set of arcs in the undirected graph G.

A set of jobs $\{1, 2, \dots, v\}$, needs to undergo processing on several sync-workstations $\{1, 2, \dots, w\}$, and each sync-workstation requires one sync-part. Since sync-workstations are not adjacent, there is an interval time $\tau_r (r \in \{1, 2, \dots, w-1\})$ if a job travels on the paced mixed-model line from sync-workstations r and $r+1$. Thus, if to assemble the same job, its start time on sync-workstation $r+1$ is τ_r time later than on sync-workstation r. The takt time of the assembly line is T, which refers to the pace at which a job is produced.

The total number of sync-parts required is $s = v \times w$. The set of sync-parts is denoted as $\{1, 2, \dots, s\}$. Within the set of sync-parts, the first w sync-parts correspond to the sync-parts for the first job; similarly, the $w+1$ to $2w$ sync-parts correspond to the sync-parts for the second job, and so on. q_m represents the size of sync-part m. We define a given binary variable sp_{mj} (0 or 1) which indicates that supplier j can provide sync-part m if $sp_{mj} = 1$, otherwise 0. Assume each sync-part is associated with a single supplier.

A sync-part cannot be divided. There are a finite number of homogeneous vehicles, denoted as $\{1, 2, \dots, K\}$, with the same capacity $Q (q_m \le Q, m \in \{1, 2, \dots, s\})$. Due to loading and unloading times being relatively short compared to transportation times, we ignore them in our model. The vehicles depart from the manufacturer and transport different sync-parts produced by the same or different suppliers to the manufacturer. The vehicles travel multiple trips and have a maximum number of trips which is equal to s.

The objective of the problem is to find a coordinated schedule of car sequencing and vehicle routing to minimize the makespan, i.e., $C_{\max}$, which represents the duration from the beginning of transportation until the completion of assembly. The further assumptions are considered as follows.

1. Each vehicle can travel multiple trips.
2. Each supplier is visited by a vehicle in a single trip no more than once.
3. Suppliers hold sufficient inventory of parts, and the loading and unloading time for vehicles is ignored.
4. A vehicle can carry multiple and different parts.
5. Vehicles travel at a constant speed.

3.2. Model formulation

Indices:

- k: index of vehicle, $k = \{1, 2, \dots, K\}$
- i, j: index of vertex, $i, j \in \{0, 1, 2, \dots, n, n+1\}$
- r: index of sync-workstation, $r \in \{1, 2, \dots, w\}$
- g: index of trip, $g \in \{1, 2, \dots, s\}$
- m: index of sync-part, $m \in \{1, 2, \dots, s\}$
- p: index of job, $p \in \{1, 2, \dots, v\}$

Parameters:

- t_{ij}: travel time of arc (i, j)
- q_m: the size of sync-part m
- Q: the capacity of a homogeneous vehicle
- T: takt time of the mixed-model assembly line
- M: a large positive number
- τ_r: the interval time between sync-workstation r and sync-workstation
- sp_{mj}: 1 if supplier j provides sync-part m, 0 otherwise

Variables:

- $x_{ijkg} \in \{0, 1\}$: 1 if vehicle k travels arc (i, j) in trip g, 0 otherwise
- $y_{mkg} \in \{0, 1\}$: 1 if vehicle k loads sync-part m in trip g, 0 otherwise
- $z_{jkg} \in \{0, 1\}$: 1 if vehicle k travels vertex j in trip g, 0 otherwise
- $\theta_{pp'} \in \{0, 1\}$: 1 if job p is assigned to position p', 0 otherwise, $p' \in \{1, 2, \dots, v\}$
- A_i^{kg}: the arrive time of vehicle k at vertex i in trip g
- C_0: the production start time
- d_m: the due date of sync-part m
- $C_{\max}$: the makespan from the start of transportation to the completion of production

The mathematical model of our problem is formulated as follows:

$$\text{Min } C_{\max} \tag{1}$$

$$\sum_{i=0, i\neq j}^{n+1} x_{ijkg} \le 1, \ j = 0, 1, \dots, n+1, k = 1, 2, \dots, K, \ g = 1, 2, \dots, s \tag{2}$$

$$\sum_{j=0, j\neq i}^{n+1} x_{ijkg} \le 1, \ i = 0, 1, \dots, n+1, k = 1, 2, \dots, K, g = 1, 2, \dots, s \tag{3}$$

$$\sum_{i=0, i\neq i'}^{n} x_{ii'kg} - \sum_{j=1, j\neq i'}^{n+1} x_{i'jkg} = 0, \ i' = 1, 2, \dots, n, k = 1, 2, \dots, K, g = 1, 2, \dots, s \tag{4}$$

$$x_{ijkg} + x_{jikg} \le 1, \ i, j = 0, 1, \dots, n+1, k = 1, 2, \dots, K, g = 1, 2, \dots, s \tag{5}$$

$$\sum_{j=1}^{n} x_{0jkg} \ge x_{ijkg}, \ i, j = 0, 1, \dots, n+1, k = 1, 2, \dots, K, g = 1, 2, \dots, s \tag{6}$$

$$\sum_{j=1}^{n+1} x_{0jkg} = \sum_{i=0}^{n} x_{i,n+1,\ \mathrm{kg}}, \ k = 1, 2, \dots, K, g = 1, 2, \dots, s \tag{7}$$

$$\sum_{m=1}^{s} y_{mkg} q_m \le Q,\ k=1,2,\dots,K, g=1,2,\dots,s \tag{8}$$

$$A_i^{kg}+t_{ij}-A_j^{kg} \le (1-x_{ijkg})M, i,j=0,1,\dots,n+1, k=1,2,\dots,K, g=1,2,\dots,s \tag{9}$$

$$A_j^{kg} \le A_0^{k,g+1},\ j=1,2,\dots,n+1, k=1,2,\dots,K, g=1,2,\dots,s-1 \tag{10}$$

$$A_i^{kg} \ge 0,\ i=0,1,\dots,n+1, k=1,2,\dots,K, g=1,2,\dots,s \tag{11}$$

$$\sum_{m=1}^{s} y_{mkg} sp_{mj} \ge z_{jkg},\ j=1,2,\dots,n, k=1,2,\dots,K, g=1,2,\dots,s \tag{12}$$

$$z_{jkg} \ge \frac{1}{s}\sum_{m=1}^{s} y_{mkg} sp_{mj},\ j=1,2,\dots,n, k=1,2,\dots,K, g=1,2,\dots,s \tag{13}$$

$$z_{jkg} = \sum_{i=0,i\ne j}^{n} x_{ijkg},\ j=1,2,\dots,n, k=1,2,\dots,K, g=1,2,\dots,s \tag{14}$$

$$M\sum_{m=1}^{s} y_{mkg} \ge \sum_{m=1}^{s} y_{m,k,g+1},\ k=1,2,\dots,K, g=1,2,\dots,s-1 \tag{15}$$

$$d_m-(A_0^{kg}+\sum_{i=0}^{n+1}\sum_{j=0}^{n+1} x_{ijkg}t_{ij})y_{mkg} \ge 0, m=1,2,...,s, k=1,2,...,K, g=1,2,\dots,s \tag{16}$$

$$\sum_{g=1}^{s}\sum_{k=1}^{K} y_{mkg} = 1,\ m=1,2,\dots,s \tag{17}$$

$$\sum_{p=1}^{v} \theta_{pp'} = 1,\ p'=1,2,\dots,v \tag{18}$$

$$\sum_{p'=1}^{v} \theta_{pp'} = 1,\ p=1,2,\dots,v \tag{19}$$

$$d_{w\times(p-1)+r} = C_0+\sum_{r'=0}^{r-1}\tau_{r'}+(\sum_{p'=1}^{v} p'\theta_{pp'}-1)T, p=1,2,\dots,v, r=1,2,\dots,w \tag{20}$$

$$C_0 > 0 \tag{21}$$

$$C_{\max} \ge d_m + T,\ m=1,2,\dots,s \tag{22}$$

$$x_{ijkg} \in \{0,1\}, i,j=0,1,\dots,n+1, k=1,2,\dots,K, g=1,2,\dots,s \tag{23}$$

$$y_{mkg} \in \{0,1\},\ m=1,2,\dots,s, k=1,2,\dots,K, g=1,2,\dots,s \tag{24}$$

$$z_{jkg} \in \{0,1\},\ j=1,2,\dots,n+1, k=1,2,\dots,K, g=1,2,\dots,s \tag{25}$$

$$\theta_{pp'} \in \{0,1\},\ p,p'=1,2,\dots,v \tag{26}$$

The objective function (1) minimizes the makespan of vehicle routing and assembly. Constraints (2)–(3) ensure that the number of entries and exits for each vertex in each trip is less than or equal to 1.

The network flow conservation is imposed by Constraints (4). Constraints (5) eliminate isolated cycles. Constraints (6) impose restrictions on the departure of vehicles from the manufacturer. Constraints (7) limit the times of entries and exits for the manufacturer in a single trip. Constraints (8) take into account vehicle capacity constraints. Constraints (9) define the arrival time of vehicles. Constraints (10) establish the chronological order of distinct trips for the same vehicle. Constraints (11) ensure that the arrival time of vehicles is greater than 0.

Constraints (12)–(14) establish the sequence of vertices visited by vehicles, allowing for the same supplier to be visited in different trips by the same vehicle. Constraints (15) avoid creating empty trips. Constraints (16) ensure that the time for vehicles to return to the manufacturer satisfies the due date of sync-parts. Constraints (17) limit the number of times each sync-part is transported.

Constraints (18)–(19) establish a strict one-to-one mapping between jobs and positions. Constraints (20) specify the due date for each sync-part. Constraints (21) require the production start time to be greater than 0. Constraints (22) define the optimization objective of $C_{\max}$. Constraints (23)–(26) define the range of values for decision variables.

Note that Constraints (16) are a series of non-linear equations. In order to build a MILP model, we linearize Constraints (16) here. We first introduce two auxiliary variables ω_{mkg} and v_{ijmkg}, and let $\omega_{mkg} = A_0^{kg} y_{mkg}$, $v_{ijmkg} = x_{ijkg} y_{mkg}$. Then replacing non-linear terms by using the two auxiliary variables, constraints (16) are linearized as follows:

$$\omega_{mkg} \le M y_{mkg},\ m=1,2,\dots,s, k=1,2,\dots,K, g=1,2,\dots,s \tag{27}$$

$$\omega_{mkg} \le A_0^{kg},\ m=1,2,\dots,s, k=1,2,\dots,K, g=1,2,\dots,s \tag{28}$$

$$\omega_{mkg} \ge A_0^{kg} - M(1-y_{mkg}),\ m=1,2,\dots,s, k=1,2,\dots,K, g=1,2,\dots,s \tag{29}$$

$$v_{ijmkg} \le x_{ijkg}, i,j=0,1,\dots,n+1, m=1,2,\dots,s, k=1,2,\dots,K, g=1,2,\dots,s \tag{30}$$

$$v_{ijmkg} \le y_{mkg}, i,j=1,2,\dots,n+1, m=1,2,\dots,s, k=1,2,\dots,K, g=1,2,\dots,s \tag{31}$$

$$v_{ijmkg} \ge x_{ijkg}+y_{mkg}-1, i,j=1,2,..,n+1, m=1,2,..,s, k=1,2,..,K, g=1,2,..,s \tag{32}$$

$$v_{ijmkg} \in \{0,1\}, i,j=1,2,\dots,n+1, m=1,2,\dots,s, k=1,2,\dots,K, g=1,2,\dots,s \tag{33}$$

$$d_m-\omega_{mkg}-\sum_{i=0}^{n+1}\sum_{j=0}^{n+1} v_{ijmkg}t_{ij} \ge 0, m=1,2,\dots,s, k=1,2,\dots,K, g=1,2,\dots,s \tag{34}$$

Table 1
Interval time between each pair of workstations.

Workstation pair	τ_1(M1,M2)	τ_2(M2,M3)	τ_3(M3,M4)
Interval time	15	20	15

Table 2
Sync-part No.

	J_1	J_2	J_3	J_4
M1	1	5	9	13
M2	2	6	10	14
M3	3	7	11	15
M4	4	8	12	16

Table 3
Sync-parts supplied by different suppliers.

Suppliers	Sync-part No.
1	1, 7, 8, 9, 12, 13
2	3, 5
3	2, 4, 6, 10, 14, 15
4	11, 16

3.3. An illustrative example

An example is given for illustration. There are 4 jobs that need to be processed in 4 distinct workstations (M1, M2, M3, M4) on a paced mixed-model assembly line. The takt time T is set to 5, which means that the processing time for each job at each workstation is 5. Due to the dispersion of different workstations, there is a time interval between every pair of workstations when processing the same job, as shown in Table 1. Since each workstation requires one sync-part, a total of 16 sync-parts are needed for these jobs in these sync-workstations, which is numbered in Table 2. Table 3 illustrates the relationships between suppliers and Sync-parts. We assume that all sync-parts have the same size 20.

The sync-parts are transported by two homogeneous vehicles, each with a capacity of 100. Table 4 presents the travel times of transportation.

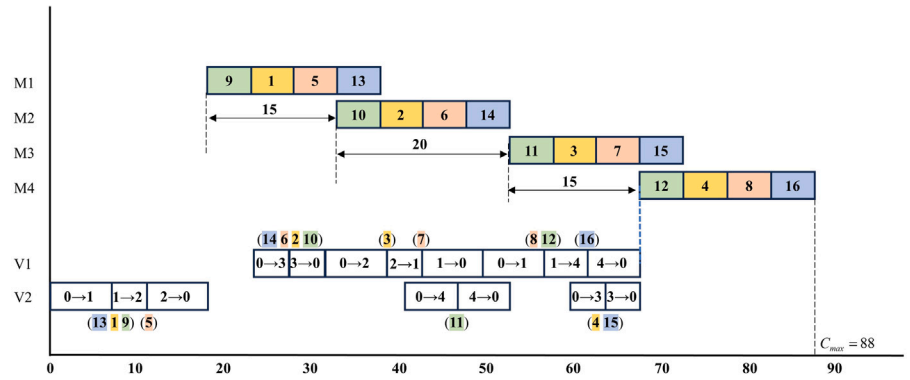


Fig. 2. Gantt chart of a feasible solution.

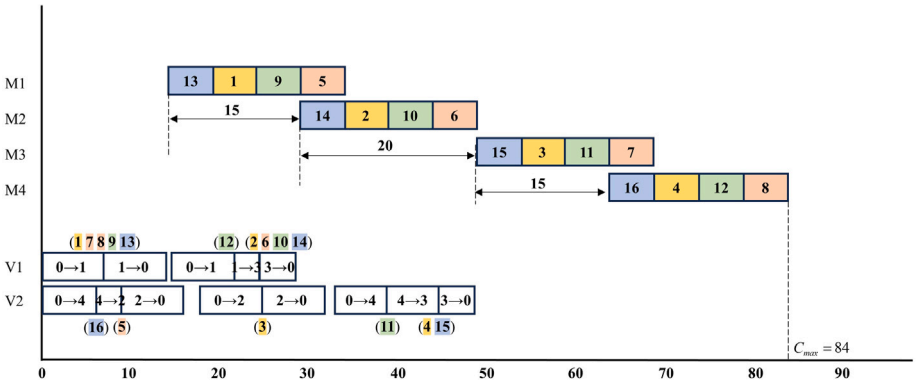


Fig. 3. Gantt chart of the optimal solution.

Table 4
Travel times between suppliers.

Vertex	0	1	2	3	4
0	0	4	5	6	4
1	4	0	4	6	5
2	5	4	0	4	4
3	6	6	4	0	6
4	4	5	4	6	0

Fig. 2 depicts the Gantt chart of a feasible solution. For each trip, it can be verified that the total load does not exceed the max capacity $Q = 100$, which satisfies the capacity constraint. Regarding vehicle routing, vehicle 1 visits suppliers by the sequence of 0-3-0-2-1-0-1-4-0. It indicates that the vehicle has three trips. In the first trip, it starts from the manufacturer (i.e., Vertex 0), visits Supplier 3, and loads sync-parts 14, 6, 2, and 10, before finally moving back to the manufacturer. Similarly, vehicle 2 has the sequence of 0-1-2-0-4-0-3-0. Regarding sequencing, the jobs are processed on the mixed-model assembly line by the sequence of 3-1-2-4. By checking the Gantt chart, it is observed that all sync-parts of a job are delivered prior to processing the job on the mix-model assembly line. The objective $C_{\max}$ of the feasible solution is 88.

Fig. 3 presents the Gantt chart of the optimal solution. It can be observed that both the vehicle capacity constraint and sync-part due date constraint are met. The sequence that vehicle 1 visits suppliers by 0-3-0-2-1-0-1-4-0, while the sequence for vehicle 2 is 0-1-2-0-4-0-3-0. The optimal solution achieves a $C_{\max}$ of 84.

4. Algorithm design

Due to the complexity of vehicle routing problems, numerous heuristics are commonly used (Hashemi Doulabi et al., 2020; Hemmelmayr et al., 2012; Zhen et al., 2020). Recently, ALNS performs successfully in various vehicle routing problems (Avci & Avci, 2019; François et al., 2019; Kuhn et al., 2021; Ropke & Pisinger, 2006). It incorporates the "destroy-repair" method and adaptive strategies, which facilitate escaping local optima. The multiple start strategy has been found as a complementary approach to heuristics, yielding favorable outcomes (Martí et al., 2013). Inspired by Pan et al. (2021), we propose a new heuristic algorithm (called MS-ALNS-VND) that incorporates multiple start strategy, ALNS and VND to address our ICSVRP problem.

Our proposed MS-ALNS-VND algorithm contributes to the field in the following aspects:

1. We propose an integrated search framework of MS, ALNS, and VND. This framework combines the strengths of the three search methods, leading to improved search capability, solution quality, and robustness. It offers a powerful approach for solving complex optimization problems efficiently.
2. We introduce a new two-segment encoding scheme that determines an integrated schedule of transportation and production. The production encoding part is a one-dimensional array, and the transportation encoding part is a three-dimensional array. The transportation coding part clearly reflects the joint decisions of sync-part assignment to vehicles, vehicle trips, and vehicle routing in each trip. Specially, the new encoding scheme reverses the order of transportation and production, i.e., first production then transportation, so that the hard constraint on sync-part due dates can be ignored.
3. Several effective operators are designed for the problem. These operators facilitate the search for solutions by effectively exploring the solution space and enabling the discovery of high-quality solutions in a more efficient manner. For example, we propose a swap operator that swaps the job with the longest waiting time. We propose several targeted destroy operators tailored to the problem at hand.

4.1. The procedure of MS-ALNS-VND

The detailed procedure of the MS-ALNS-VND is shown in Algorithm 1. At the beginning, an initial solution set S is constructed using a multi-start strategy and an initial solution construction method. A solution x_q is randomly chosen from the set S. Then, using swap operators, destroy and repair operators, and VND local search algorithm one by one, a new solution x_t is conducted. If solution x_t is better than solution x_q, the solution x_q is replaced by x_t; otherwise, it is replaced based on the simulated annealing criteria. If the total number of generations in which a solution does not show improvement reaches the specific threshold $B_{no_improve}$, it will be replaced by the global optimal solution x_g. The loop continues until the terminal condition is met.

Algorithm 1: MS-ALNS-VND

Input: P_{size}, T_0, k_0, $B_{no_improve}$;
Output: Solution x_s
Construct initial solution set S with the size of P_{size}
$NUS_q = 0 (q = 1, 2, ..., P_{size})$
Choose the optimal solution x_g as global solution from set S
while *not met terminal condition* **do**
  Randomly choose a solution x_q from S
  Use swap operators to generate a solution, $x_t = Swap(x_q)$
  Use destroy and repair operators to generate a solution, $x_t = DestroyRepair(x_t)$
  Use VND local search to generate a solution, $x_t = VND(x_t)$
  if $C_{\max}(x_t) < C_{\max}(x_q)$ **then**
    $x_q = x_t$
    if $C_{\max}(x_t) < C_{\max}(x_g)$ **then**
      $x_g = x_t$
      $NUS_q = 0$
    end
  else
    $NUS_q = NUS_q + 1$
    Generate a random number r
    if $r < exp(-(x_t - x_q)/T_0)$ **then**
      $x_q = x_t$
    end
  end
  if $NUS_q \geq B_{no_improve}$ **then**
    $x_q = x_g$
  end
  $T_0 = T_0 \times k_0$
  Update score, weight, and perturbation rate for operators
end

4.2. Two-segment reversed encoding and its decoding

In order to address the integrated optimization of sync-part supply vehicle routing and car sequencing, a new two-segment reversed encoding scheme is proposed. Specially, we reverse the order of two parts, i.e., first car sequencing then sync-part supply vehicle routing. In this manner, the hard constraint on sync-part due dates can be easily addressed, and the reversed solution can be easily transformed into the original solution.

Therefore, the proposed two-segment encoding scheme consists of production and transportation parts. As shown in Fig. 4, an example with 4 jobs (16 sync-parts) is given. The production encoding part is represented as a one-dimensional array. It indicates the jobs are processed on the mixed-model assembly line by the sequence of 4-2-1-3.

The transportation encoding part is represented as a three-dimensional array, which clearly reflects the joint decisions of sync-part assignment to vehicles, vehicle trips and vehicle routing in each trip. For example, sync-parts 16, 8, 12, 7, 3, 14, 6, 2, 10 are assigned to vehicle 1. Vehicle 1 will make three trips to deliver these sync-parts. In trip 1, it first visits supplier 4 to pick up sync-part 16 and then move to supplier 1 for sync-parts 8 and 12.

As shown in Fig. 5, we first decode the production part. Given that the four sync-workstations are non-consecutive on the mixed-model assembly line, there are time intervals between sync-workstations. For example, sync-workstation 1 and 2 have a time interval of 15. Then, the production schedule can be decoded with the job priority of 4-2-1-3.

Next, we decode the transportation part. For vehicle 1, it first visits supplier 4 to pick up sync-part 16 and then moves to supplier 1 for sync-parts 8 and 12. Note that sync-part 12 is completed at time 20. Thus, this trip begins at time 20 and returns to the manufacturer at 38. This constraint guarantees that sync-part 12 can be delivered to the manufacturer before its due date. Then its second trip visits two suppliers and carries two sync-parts, while its third trip visits one supplier to pick up four sync-parts. The same applies to vehicle 2, and finally, vehicle 2 returns to the manufacturer, concluding the decoding process. The makespan $C_{\max}$ is calculated as 88.

By reversing the decoded schedule, we can obtain the final schedule for use, which is identical to the solution shown in Fig. 2.

4.3. Initial solution construction

The initial solution is constructed by employing the regret heuristic (Ropke & Pisinger, 2006). Initially, a job sequence of production is randomly generated. Upon the generated production solution, the corresponding due dates of sync-parts are established. Subsequently, we utilize the regret heuristic to construct the routing solution. The regret heuristic inserts the sync-parts by a look-ahead scheme, which is quite effective for solving VRP problem. Let ΔC_m^r be the lowest additional cost (additional time) incurred by inserting sync-part m into trip r. If sync-part m cannot be inserted into trip r, ΔC_m^r is set to positive infinity. Regret heuristic selects sync-part m with the highest regret value (RV):

$$RV_m = \sum_{j=2}^{k} (\Delta C_m^{r_j} - \Delta C_m^{r_1}) \tag{35}$$

where r_j represent the trip for which sync-part m has the jth lowest insertion cost. Note that each vehicle is initialized with an empty trip, and regret heuristic always maintains an empty trip at the end of each vehicle. The parameter k in RV_m can be varied to increase the number of steps for look-ahead. The corresponding regret heuristic is also denoted as regret-k.

Specifically, regret-1 is defined as a greedy insertion that selects sync-part m with the lowest $\Delta C_m^{r_1}$. In our implementation, we adopt regret-1 to construct an initial solution. Regret heuristic is also utilized for repair operations in the proposed algorithm. The pseudocode of the initial solution construction is shown in Algorithm 2.

Algorithm 2: Initial solution construction

Input: job set P, sync-part set L_1
Output: initial solution x_q
An empty solution x_q
$S = A$ random job sequence
Generate due date of each sync-part in S
while L_1 *is not empty* **do**
  for $m \in L_1$ **do**
    Calculate the RV_m
  end
  Insert the sync-part m with minimum RV_m into x_q
  Remove m from L_1
end

4.4. Multiple start strategy

To enhance the algorithm efficiency, we employ multiple start strategy, which can be usually categorized into two types: memory-based and memoryless strategy (Martí et al., 2013). The memory-based strategy uses a set of memories to guide the search process, for example, adaptive memory programming strategy (Dhouib et al., 2010; Villegas et al., 2010). The memoryless strategy, on the other hand, explores the solution space by randomly generating multiple initial solutions and running a local search algorithm.

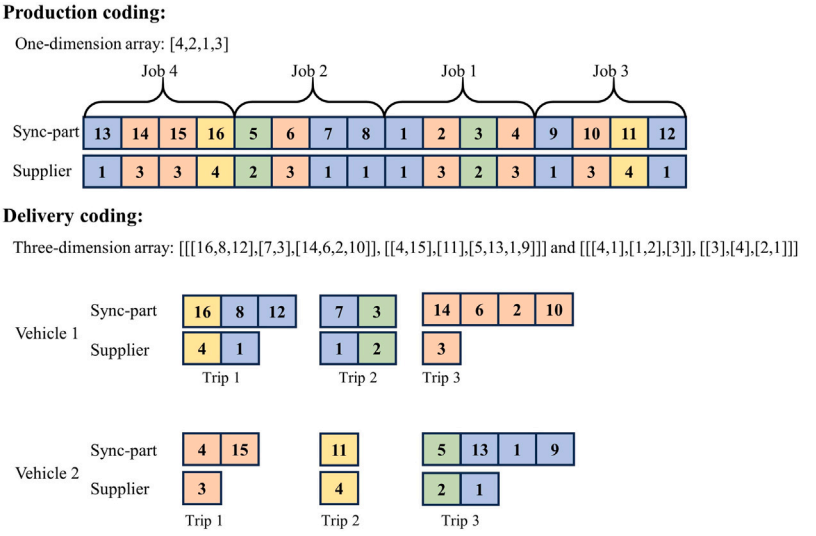


Fig. 4. The two-segment reversed encoding scheme.

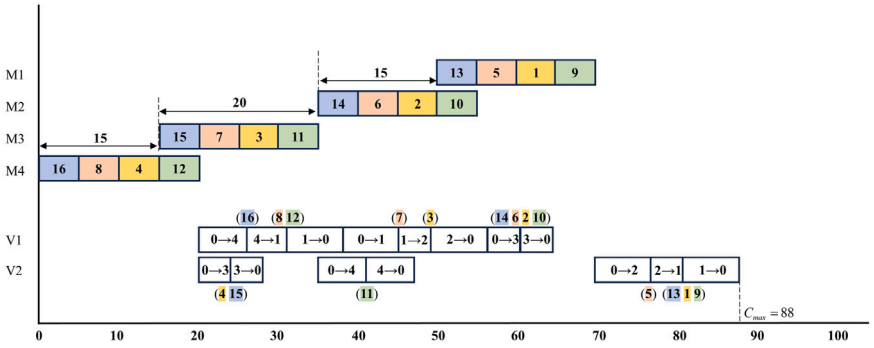


Fig. 5. The Gantt chart of the reversed solution.

The memoryless strategy has been successfully used in addressing many VRP problems with considerable computing time (Duhamel et al., 2011; Han & Chu, 2016). Consequently, we employ the memoryless strategies in this study. Specifically, at the beginning of the algorithm, we simultaneously generate several initial solutions and store them in a set S, as shown in Algorithm 1. In each iteration, a solution $x_q \in S$ is randomly chosen for subsequent operations, including destroy, repair, and local search. After completing a series of search operations, the algorithm evaluates whether the solution x_q has improved. If the solution does not show improvement for $B_{no_improve}$ consecutive iterations, it will be replaced by the optimal solution to increase the search opportunities for a better solution.

4.5. Swap operators

In each iteration, the algorithm randomly selects a solution x_q and executes the swap operation. A swap operator so is randomly chosen from the swap operators set SO, and it is applied to the chosen solution x_q. If the new solution x_t is superior to the chosen solution x_q, let $x_q = x_t$; if the new solution x_t is superior to the global solution x_g, let $x_g = x_t$. The swap process is repeated st times. The pseudocode is described in Algorithm 3.

We devise two swap operators for the exploration of job sequence. The first operator so_1 performs a random swap, which randomly selects two jobs in the encoding and exchanges their positions. Subsequently, the operator collects these sync-parts corresponding to the swapped jobs in a list L_2. Then the operator applies the RI rule to reinsert these sync-parts.

Algorithm 3: Swap operation

Input: global solution x_g, incumbent solution x_q, swap times st, swap operators set SO
Output: global solution x_g, incumbent solution x_q

```
for i ∈ {1, 2, ..., st} do
    Randomly choose a swap operator so from SO
    Swap two jobs of x_q using so and remove the corresponding sync-parts to an
      empty list L_2 to generate a new solution x_t
    while L_2 is not empty do
        Insert sync-part m using greedy insertion to update x_t
        Remove m from L_2
    end
    if C_max(x_t) < C_max(x_q) then
        x_q = x_t
        if C_max(x_t) < C_max(x_g) then
            x_g = x_t
        end
    end
end
```

The second operator so_2 is proposed to swap the job with the longest waiting time. Firstly, the waiting time wt_m caused by each sync-part is computed. Subsequently, the job with the longest waiting time is selected and swapped with a random job.

The calculation of wt_m is illustrated in Fig. 6. Vehicle V1 has three trips, and only trip 1 incurs waiting time as $wt_{14}, wt_6, wt_2, wt_{10} = 23$. And other sync-parts in this trip do not need to wait. Vehicle V2 also has three trips. Since The begin time of trip 1 is 0, we have $wt_{13}, wt_1, wt_9, wt_5 = 0$. Trip 2 begins at 40, resulting in $wt_{11} = 40 - 18 = 22$. Similarly, wt_4, wt_{15} is known as 7. Let the waiting time of a job equal

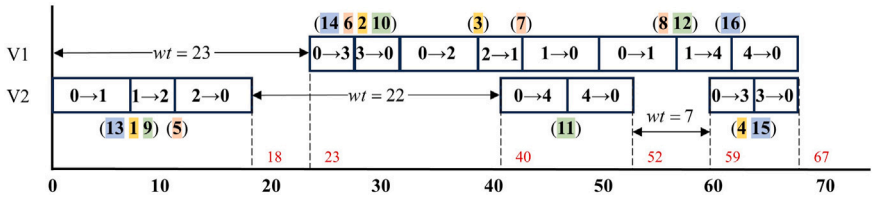


Fig. 6. The example of waiting time calculation.

to the total waiting times of its sync-parts. Then the waiting times of jobs 1, 2, 3, and 4 are calculated as 30, 23, 45, and 30. Therefore, job 3 is selected with the longest waiting time.

4.6. Destroy and repair operators

The proposed MS-ALNS-VND employs six destroy operators in total. The first and second operators introduce random destructions based on sync-part and trip, respectively. We design the third and fourth operators based on Shaw destroy operator using a related-rule destruction approach (Shaw, 1997).

We further propose two new operators for our algorithm, namely, part waiting-based destroy and trip efficiency-based destroy. The part waiting-based destroy operator destroys sync-part one by one based on the longest waiting time rule. The trip efficiency-based destroy operator employs trip-efficiency-rule and related-rule.

4.6.1. Random destroy operator

Random destroy operators simply destroy elements that are selected at random. The first operator randomly destroys N_1 sync-parts, while the second operator destroys N_2 trips. These two operators tend to generate perturbed solutions to diversify the search.

4.6.2. Shaw destroy operator

Destroying similar elements can lead to improved solutions. The Shaw destroy operator employs a related-rule that describes the degree of similarity between elements (Shaw, 1997). In this study, we employ the related-rule to generate two Shaw destroy operators.

In the first Shaw destroy operator, we destroy one sync-part in each iteration. This process is iterated until the number of destroyed sync-parts reaches N_1. The detailed procedure is outlined in Algorithm 4. The parameters α, β, and d are set as 0.5, 0.5, and 6, respectively.

Algorithm 4: The first Shaw destroy operator

Input: incumbent solution x_q, parameters α, β and d, the maximum number N_1
Output: destroyed sync-part list L_3, destroyed solution x_q

Random select a sync-part, destroy it from the solution x_q and put it in the sync-part list L_3
while $|L_3| < N_1$ **do**
  Randomly select a sync-part m' in L_3
  for *each sync-part m in solution x_q* **do**
    $dis_{m',m}$ = the distance between the supplier of part m' and the supplier of part m
    $du_{m',m}$ =the gap between the due date of part m' and the due date of part m
    $z_{m',m} = \alpha \times dis_{m',m} + \beta \times du_{m',m}$
  end
  Sort sync-parts according to $z_{m',m}$ in non-decreasing order and put them in list B
  Choose a random number χ between 0 and 1
  $pos = \lceil |B| \times \chi^d \rceil$
  Select sync-part j associated with the $z_{m',m}$ value at position pos in B and remove it from solution x_q
  $L_3 = L_3 \cup \{m\}$
end

In the second Shaw destroy operator, we adopt a similar procedure to the first Shaw destroy operator except for destruction at the trip level. The specific procedure is as shown in Algorithm 5.

Algorithm 5: The second Shaw destroy operator

Input: incumbent solution x_q, d, the maximum number N_2
Output: destroyed trip list L_3, destroyed solution x_q

Random select a trip, destroy it from the solution x_q and put it in the trip list L_3
while $|L_3| < N_2$ **do**
  Randomly select a sync-part r' in L_3
  for *each trip r in solution x_q* **do**
    Calculate the gravity center of r and r' as g_r and $g_{r'}$ respectively
    $z_{r',r}$ =the distance of g_r and $g_{r'}$
  end
  Sort trips according to $z_{r',r}$ in non-decreasing order and put them in list B
  Choose a random number χ between 0 and 1
  $pos = \lceil |B| \times \chi^d \rceil$
  Select trip r associated with the $z_{r',r}$ value at position pos in B and remove it from solution x_q
  $L_3 = L_3 \cup \{r\}$
end

4.6.3. Part waiting-based destroy operator

The part waiting-based destroy operator is proposed based on the concept of waiting time wt_m introduced in Section 4.5. In each destruction, the sync-part with the longest waiting time is selected for destruction, and this process is repeated until N_1 sync-parts are destroyed.

4.6.4. Trip efficiency-based destroy operator

The trip efficiency-based destroy operator is designed based on the proposal of trip efficiency with the related-rule (Shaw, 1997).

We calculate the average transportation time ATT_r for each non-empty trip in the solution to assess the trip efficiency: ATT_r = the travel time of trip r/ the number of sync-parts in trip r. The probability of selecting trip r is set as $ATT_r / \sum ATT_r$. We first select and destroy a trip using the probability, and then use the related-rule for the rest of destruction. The specific process is outlined in Algorithm 6.

Algorithm 6: Trip efficiency-based destroy operator

Input: incumbent solution x_q, d
Output: destroyed trip list L_3, destroyed solution x_q

Destroy the trip r with the maximum ATT_r
$L_3 = \emptyset$
Add all sync-parts belonging to the trip r to L_3
while $|L_3| < N_1$ **do**
  Randomly select a sync-part m' in L_3
  for *each sync-part m in solution x_q* **do**
    $dis_{m',m}$ = the distance between the supplier of part m' and the supplier of part m
    $du_{m',m}$ =the gap between the due date of part m' and the due date of part m
    $z_{m',m} = \alpha \times dis_{m',m} + \beta \times du_{m',m}$
  end
  Sort the $z_{m',m}$ in non-decreasing order and put them in list B
  choose a random number χ between 0 and 1
  $pos = \lceil |B| \times \chi^d \rceil$
  Select sync-part m associated with the $z_{m',m}$ value at position pos in B and remove it from solution x_q
  $L_3 = L_3 \cup \{m\}$
end

4.6.5. Repair operators

We perform repair operations using regret heuristic. In order to increase the randomness of repaired solutions, we vary k values of

regret heuristic from $1, 2, 3$, and $k_{\max}$ to generate four repair operators, namely, regret-1, regret-2, regret-3, regret-$k_{\max}$, where $k_{\max}$ denotes the number of trips that can be inserted by sync-parts.

4.7. Roulette wheel selection for operators

All the destroy and repair operators are selected by probability function. Each operator j in the operator set OP is given a dynamic weight of wo_j. The probability of selecting operator j is by the function of $wo_j / \sum_{i \in OP} wo_i$. Initially, each operator is assigned with the same weight. In each iteration of the ALNS part, the selected pair of operators (destroy and repair operators) receive the same score of ρ_1 if it results in a new global best solution and the same score of ρ_2 if it produces a solution better than the incumbent solution. If these operators do not improve the solution, no score is awarded.

We partition the algorithm into multiple segments, each comprising 100 iterations. The weights of the operators are updated after every segment (100 iterations). The weights are updated by the following formula:

$$wo_{i,s+1} = (1-\varepsilon) wo_{i,s} + \varepsilon \frac{\pi_{i,s}}{\max\{1, \theta_{i,s}\}} \tag{36}$$

where $wo_{i,s}$ denotes the weight of operator i in the sth segment; $\pi_{i,s}$ represents the score obtained by operator i in the sth segment; $\theta_{i,s}$ denotes the number of times operator i is invoked in the sth segment. And ε is the coefficient controlling the dynamic weight adjustment, where $\varepsilon \in (0, 1)$. In this study, we set ε to 0.92.

4.8. Dynamic perturbation strength

In our algorithm, we introduce dynamic perturbation strength to the swap operations and destroy operations. The perturbation strength is defined as the ratio of destroyed orders, trips or swap times.

We introduce a counter $NoImp$, representing no improvement for the consecutive $NoImp$ iteration. If no better global solution is obtained during an iteration, the $NoImp$ increases by 1. Conversely, $NoImp$ is reset to 0. At the beginning of the algorithm, the swap times $st = \gamma \times v$, the number of destroyed sync-parts $N_1 = \gamma \times v \times w$, and the number of destroyed trips $N_2 = \gamma \times n_{trips}$ are set as initial values. v, w, and n_{trips} denote the number of jobs, the number of sync-workstations, and the number of all trips in the solution.

During the execution of the algorithm, these three values st, N_1 and N_2 are dynamically adjusted as $\min\{0.2, \gamma + 0.0012 NoImp\} \times v$, $\min\{0.2, \gamma + 0.0012 NoImp\} \times v \times w$ and $\min\{0.2, \gamma + 0.0012 NoImp\} \times n_{trips}$, respectively.

In order to enhance the global search capability of our algorithm, we also introduce the annealing component of the simulated annealing (SA) algorithm (Rutenbar, 1989). By doing so, we introduce randomness into the search space, allowing for accepting solutions worse than the current one with a certain probability. This introduction adds an exploratory capability to the algorithm, helping to escape local optima and better explore the space for global optimal solutions. Let T_0 and k_0 denote the temperature and the annealing rate, respectively. In this study, we set $T_0 = 35 \times v \times w$ and $k_0 = 0.97$.

4.9. VND local search

The local search is implemented by VND. Inspired by the local search operators used by Pan et al. (2021), we design six local search operators for the VND. The first operator ls_1 randomly swaps the two suppliers in the same trip, the sync-parts belonging to these two suppliers are also exchanged. The second operator ls_2 randomly selects two trips from the same vehicle and exchanges two suppliers from the trips. The third operator ls_3 randomly selects two trips from different vehicles and exchanges them. The fourth operator ls_4 randomly destroys a trip and then repairs it using a random repair operator in Section 4.6.5. The fifth operator ls_5 destroys and repairs two consecutive trips by a random repair operator. The sixth operator ls_6 destroys two trips from two different vehicles and then repairs them by a random repair operator. The VND procedure of the MS-ALNS-VND algorithm is depicted in Algorithm 7.

Algorithm 7: VND local search

Input: incumbent solution x_t, a set of local search operators $LS = \{ls_i, i = 1, 2, ..., 6\}$
Output: solution x_t

```
i = 1
while i ≤ 6 do
    x'_t = ls_i(x_t)
    if C_max(x'_t) < C_max(x_t) then
        x_t = x'_t
        i = 1
    else
        i = i + 1
    end
end
```

5. Numerical experiments

This section conducts numerical experiments on the parameters' configuration, algorithm comparison, and sensitivity analysis. First, we employ the Taguchi method to tune the algorithm parameters. Second, we assess the performance of the MS-ALNS-VND in comparison with the Gurobi solver, ALNS-VND, and ALNS for different sized instances. Finally, we study the effect of varying the number of vehicles on the objective of makespan.

All algorithms are implemented with Python, and the MILP model is solved with Gurobi 10.0.0. All experiments are run in a Windows system with an 11th Gen Intel(R) Core(TM) i5-11400H CPU of 2.69 GHz and 16 GB of memory.

5.1. Parameter settings

To the best of our knowledge, there are no established benchmarks for the ICSVRP problem. Therefore, we define the parameters and generate instances through empirical investigations as well as Solomon's VRP instances (Solomon, 1987). Solomon's VRP instances are categorized into three sets: C, R, and RC, representing clustered generated nodes, randomly generated nodes, and a combination of both, respectively.

t_{ij} (Travel time matrix): The set of Solomon's 25 and 50 nodes VRP instances are used to generate our instances. Instances with fewer 25 suppliers are randomly generated using Solomon's 25 nodes VRP instances, while instances with more than 25 suppliers are generated using Solomon's 50 nodes VRP instances. The distances between suppliers are calculated using the Euclidean distance, and the time matrix is obtained by dividing the distances by the constant speed V.

w (Number of sync-workstations): Based on our investigation, most electric automobile manufacturers have 4 sync-workstations.

τ_r (Interval time): The time intervals between different sync-workstations correlate with the production cycle Γ. Based on our observations, these processes are generally evenly distributed throughout the automobile assembly process. Therefore, we have the following:

$$\tau_r = \frac{\Gamma}{w-1}, r \in \{1, 2, \ldots, w-1\}$$

T (Takt time): In our study, the processing time of jobs on sync-workstations can be considered equivalent to the takt time. The takt time is set as T.

sp_{mj} (Supplier): We randomly assign different suppliers to each sync-part.

Q (Vehicle capacity): We use K homogeneous vehicles, and the capacity of each vehicle is set to Q.

q_m (Size): The size of each sync-part is generated according to the vehicle capacity. Consider four types of sync-parts with synchronized

Table 5
Parameter setting of β_r.

Parameter	Parameter set
β_1	{0.15, 0.20}
β_2	{0.08, 0.12}
β_3	{0.08, 0.10, 0.12, 0.14}
β_4	{0.10, 0.11, 0.12, 0.13, 0.14, 0.15}

Table 6
Instances of three sizes.

Small		Medium		Large	
Instance	*RT*	Instance	*RT*	Instance	*RT*
C2-2-1	4	C30-10-2	120	C70-25-3	420
R2-2-1	4	R30-10-2	120	R70-25-3	420
RC2-2-1	4	RC30-10-2	120	RC70-25-3	420
C3-2-1	6	C30-10-3	180	C70-25-4	560
R3-2-1	6	R30-10-3	180	R70-25-4	560
RC3-2-1	6	RC30-10-3	180	RC70-25-4	560
C5-4-1	10	C40-10-3	240	C85-25-4	680
R5-4-1	10	R40-10-3	240	R85-25-4	680
RC5-4-1	10	RC40-10-3	240	RC85-25-4	680
C5-4-2	20	C40-15-4	320	C85-50-5	850
R5-4-2	20	R40-15-4	320	R85-50-5	850
RC5-4-2	20	RC40-15-4	320	RC85-50-5	850
C10-5-1	20	C50-15-3	300	C100-25-5	1000
R10-5-1	20	R50-15-3	300	R100-25-5	1000
RC10-5-1	20	RC50-15-3	300	RC100-25-5	1000
C10-5-2	40	C50-15-4	400	C100-50-5	1000
R10-5-2	40	R50-15-4	400	R100-50-5	1000
RC10-5-2	40	RC50-15-4	400	RC100-50-5	1000

supply requirements. Each sync-part is highly customized, so their sizes may vary. Let $q_m = \beta_r Q$ if sync-part m belongs to sync-workstation r. Table 5 presents the values of β_r for sync-parts.

Let $V = \frac{1}{3}$, $\Gamma = 5760$, $T = 100$ and $Q = 100$. As shown in Table 6, the instances are categorized into small-, medium-, and large-sized. The instances are named as "Type v-n-K", where "Type" contains C, R and RC, v represents the number of jobs, n represents the number of suppliers, and K represents the number of vehicles. We generate 54 instances, containing 18 small-sized, 18 medium-sized, and 18 large-sized instances. The running time of each instance is set as $RT = 2{\times}v{\times}K$ seconds. The maximum running time of Gurobi solver is limited to 1800 s. Each instance is run 10 times.

5.2. Calibration of algorithm parameters

Configuring appropriate parameter values for the MS-ALNS-VND algorithm is crucial. To this end, we conduct an experimental design (DOE) involving five parameters, namely P_{size}, ρ_1, ρ_2, γ, $B_{no_improve}$, as shown in Table 7. Since each of the five parameters is tested at five levels, resulting in 5^5 possible parameter combinations, we employ Taguchi's L25 orthogonal array to determine the optimal parameter combination efficiently.

Tests are conducted on 6 instances, namely, C10-6-2, R10-6-2, RC10-6-2, C15-10-2, R15-10-2, and RC15-10-2. For each parameter combination, we conduct 10 independent replicates on each instance. We calculate $Gap = AVG(Obj_{ij} - Obj_i^{bst})/Obj_{ij} \times 100$ to assess the performance of a parameter combination, where Obj_{ij} means the objective value of ith ($i = 1, \ldots, 6$) instance's jth ($j = 1, \ldots, 10$) run, Obj_i^{bst} represents the best solution of instance i obtained across all experiments.

Based on the experimental results presented in Table 8, the parameters for the proposed MS-ALNS-VND algorithm are set as $P_{size} = 9$, $\rho_1 = 25$, $\rho_2 = 8$, $\gamma = 0.1$, $B_{no_improve} = 22$.

5.3. Algorithm comparisons

This section presents the computational results of algorithm comparisons. First, we compare MS-ALNS-VND with Gurobi solver to validate the correctness of MS-ALNS-VND on the small-sized instances. Subsequently, we perform algorithm comparisons with the ALNS-VND (Pan et al., 2021) and the ALNS (Ropke & Pisinger, 2006) to illustrate the advantages of MS-ALNS-VND on the medium- and large-sized instances.

5.3.1. Comparison with gurobi on small-sized instances

Table 9 presents the results obtained by MS-ALNS-VND and Gurobi solver. For Gurobi solver, we report whether the optimal solution is found (Yes or No), the objective value of the obtained solution (C_{Gurobi}), and the running time required for the optimal solution (T_{Gurobi}). The running time of the Gurobi solver is limited to 1800 s. Regarding MS-ALNS-VND, we report the objective value (C_{bst}) of the best solution obtained in 10 runs and its running time (T_{bst}).

It is observed that Gurobi solver solves 7 out of 18 instances to optimality within 1800 s. This observation illustrates the complexity of the studied problem as the problem size increases. In contrast, MS-ALNS-VND can also solve all 7 instances optimally in much less running time. Furthermore, it rapidly achieves better solutions on another 10 instances that Gurobi solver fails to find optimal solutions.

5.3.2. Comparison with ALNS-VND and ALNS on the medium- and large-sized instances

Since the ALNS-VND and the ALNS are not designed for the ICSVRP, we make several modifications to fit the studied problem. First, we introduce the proposed encoding and decoding scheme to both ALNS-VND and ALNS. Second, the proposed swap operators, i.e., so_1 and so_2, are used in them. Third, the local search operator ls_1 is used for the ALNS-VND. Similarly, the parameters of the two algorithms are tuned to use.

Let C_{bst} denote the best objective value and C_{avg} denote the average objective value in 10 runs for each algorithm. The indicator "Gap" is redefined as $((C_{avg}^X - C_{avg}^{MS})/C_{avg}^{MS}) \times 100$, where C_{avg}^X represents the average objective value obtained by ALNS-VND or ALNS, C_{avg}^{MS} represents the average objective value obtained by MS-ALNS-VND.

Table 10 presents the comparison results on medium-sized instances. Among the 18 medium-sized instances, MS-ALNS-VND finds half of the best solutions, while ALNS-VND finds six of them, and ALNS only finds three of them. In addition, MS-ALNS-VND achieves better C_{avg} than ALNS-VND and ALNS on 16 and 18 instances, respectively. The average gaps of ALNS-VND and ALNS with our algorithm are 3.81% and 6.05%, respectively. This demonstrates the superior performance of the MS-ALNS-VND algorithm on medium-sized instances.

Table 11 presents the comparison results on large-sized instances. Among the 18 large-sized instances, MS-ALNS-VND finds 17 best solutions, while ALNS-VND finds one of them, and ALNS finds none. In addition, MS-ALNS-VND achieves better C_{avg} than ALNS-VND and ALNS

Table 7
The parameters in MS-ALNS-VND.

Parameter	Range	Description
P_{size}	{3, 5, 7, 9, 11}	The number of solutions of multiple start strategy
ρ_1	{25, 28, 31, 34, 37}	The reward when finding better global solution in the adaptive phase of the algorithm
ρ_2	{5, 8, 11, 14, 17}	The reward when finding better solution than the incumbent solution in the adaptive phase of the algorithm
γ	{0.06, 0.08, 0.10, 0.12, 0.14}	The initial intensity of the perturbation
$B_{no_improve}$	{10, 16, 22, 28, 34}	The maximum number of consecutive non-improving iterations allowed for a single solution

Table 8
Orthogonal array and calibration results of parameters.

Experiment No.	P_{size}	ρ_1	ρ_2	γ	$B_{no_improve}$	Gap (%)
1	3	25	5	0.06	10	6.03
2	3	28	8	0.08	16	5.78
3	3	31	11	0.1	22	5.24
4	3	34	14	0.12	28	6.94
5	3	37	17	0.14	34	6.90
6	5	25	8	0.1	28	5.55
7	5	28	11	0.12	34	5.76
8	5	31	14	0.14	10	5.72
9	5	34	17	0.06	16	5.39
10	5	37	5	0.08	22	6.32
11	7	25	11	0.14	16	6.42
12	7	28	14	0.06	22	5.87
13	7	31	17	0.08	28	5.60
14	7	34	5	0.1	34	5.62
15	7	37	8	0.12	10	5.66
16	9	25	14	0.08	34	5.15
17	9	28	17	0.1	10	5.62
18	9	31	5	0.12	16	5.76
19	9	34	8	0.14	22	4.36
20	9	37	11	0.06	28	5.84
21	11	25	17	0.12	22	4.40
22	11	28	5	0.14	28	5.36
23	11	31	8	0.06	34	5.75
24	11	34	11	0.08	10	5.73
25	11	37	14	0.1	16	5.87

Table 9
Comparison with Gurobi solver on the small-sized instances.

Instance	Gurobi			MS-ALNS-VND	
	Optimal?	C_{Gurobi}	T_{Gurobi}	C_{bst}	T_{bst}
C2-2-1	Yes	**6043.15**	0.75	**6043.15**	0.32
R2-2-1	Yes	**5736.55**	1.72	**5736.55**	0.44
RC2-2-1	Yes	**6492.45**	1.13	**6492.45**	0.35
C3-2-1	Yes	**5964.67**	3.35	**5964.67**	0.01
R3-2-1	Yes	**6276.55**	10.28	**6276.55**	0.36
RC3-2-1	Yes	**7801.39**	9.82	**7801.39**	0.37
C5-4-1	No	6859.11	1800.08	**6509.11**	0.56
R5-4-1	No	6972.26	1800.13	**6278.80**	1.67
RC5-4-1	No	8032.06	1800.37	**7147.86**	0.76
C5-4-2	No	6287.80	1800.09	**5936.23**	0.79
R5-4-2	No	5961.64	1800.26	5986.19	3.69
RC5-4-2	Yes	**6432.45**	887.47	**6432.45**	4.29
C10-5-1	No	9492.11	1800.28	**7986.42**	10.69
R10-5-1	No	9668.73	1800.13	**8604.31**	3.95
RC10-5-1	No	14 450.35	1800.16	**11 817.95**	5.57
C10-5-2	No	6764.93	1800.48	**6689.64**	9.00
R10-5-2	No	7160.67	1800.22	**6731.82**	4.57
RC10-5-2	No	9010.96	1800.32	**8248.87**	11.68

The **bold** numbers represent the optimal values among these algorithms.

on 17 and 18 instances, respectively. The average gaps of ALNS-VND and ALNS with our algorithm are 4.85% and 6.83%, respectively.

It is observed that the gaps of ALNS-VND and ALNS in large-sized instances are larger than the gaps obtained in medium-sized instances, suggesting that MS-ALNS-VND performs even more effectively in large instances than in medium-sized instances. It is observed that the gaps of ALNS-VND and ALNS in large-sized instances are comparatively larger than those obtained in medium-sized instances, suggesting that MS-ALNS-VND performs even more effectively as the problem size increases. This explicitly illustrates the benefits of integrating a multi-start strategy and VND into ALNS, thereby accentuating our algorithm's advantages.

5.4. Impact of number of vehicles

The number of vehicles plays a critical role in solving real-world ICSVRP. Insufficient vehicles can result in delayed deliveries, while having too many vehicles can increase investment costs. Therefore, we conduct a sensitivity analysis experiment to examine how the number of vehicles (K) affects the objective of makespan. Each instance is run five times and their average objective value is used.

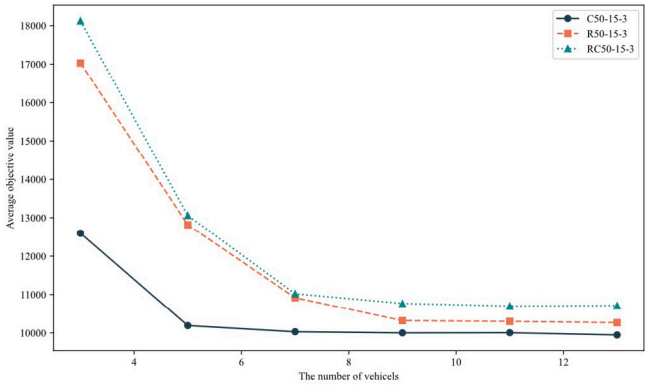


Fig. 7. The trend of the average objective value as the number of vehicles increases.

Table 12 presents the average objective values for instances C50-15-3, R50-15-3, and RC50-15-3 with different number of vehicles. Fig. 7 clearly illustrates the trend of the average objective value as the number of vehicles increases. It is evident that as the number of vehicles increases, there is a notable reduction in the average objective values. However, as the number of vehicles continues to increase, the rate of reduction becomes less pronounced, indicating diminishing marginal returns. For example, when the number of vehicles increases to 8, further increase does not result in a significant reduction in the objective value. This observation contributes to decision-making regarding vehicle procurement as it highlights the trade-off between timely delivery and physical costs.

6. A case study

The case study is based on real data from an electric automobile manufacturer located in Guangzhou, China. We select three distinct sets of production scheduling data from the manufacturer for the years 2019, 2021, and 2022. To comply with the manufacturer's data confidentiality requirements, we have anonymized the data and distilled it into several parameters relevant to our study, as presented in Table 13.

Although we have selected data from three different periods, the suppliers who provide parts to the manufacturer have remained consistent, with specific location details illustrated in Fig. 8. The purple point represents the manufacturer, which collaborates with 17 suppliers distributed within a radius of 5 kilometers. The addresses of manufacturers and suppliers are converted to latitude and longitude using the Baidu Maps API, which is then used to automatically calculate the distances and travel times between different nodes.

Moreover, the manufacturer's production plans and vehicle models varied across different years. In 2021, due to a significant production line overhaul, the cycle time was reduced to 120 s, consequently shortening the production cycle from 4320 to 2590 s. This led to a corresponding increase in output, but it also necessitated an increase in the number of vehicles required, from 6 to 9. The working hours within a planning horizon for these three periods did not vary significantly. Based on this data, the results obtained by using our algorithm are presented in Table 14. The indicator "Gap" is redefined as $((C_{\max} - C^*_{\max})/C_{\max}) \times 100$, where $C^*_{\max}$, which represents the average objective value obtained by MS-ALNS-VND. In the case studies conducted over the years 2019, 2021, and 2022, our algorithm achieved improvements of 26.84%, 23.76%, and 24.53%, respectively. This demonstrates the applicability and efficiency of the proposed MS-ALNS-VND algorithm.

The case study reveals several advantages with its application. Firstly, it demonstrates a significant reduction in makespan, leading to

Table 10
Comparison with ALNS-VND and ALNS on medium-sized instances.

Instance	MS-ALNS-VND		ALNS-VND			ALNS		
	C_{bst}	C_{avg}	C_{bst}	C_{avg}	Gap(%)	C_{bst}	C_{avg}	Gap(%)
C30-10-2	11 595.77	11 600.88	**11 207.50**	12 027.46	3.68	12 188.11	12 493.26	7.69
R30-10-2	**11 896.74**	12 507.90	12 755.28	13 163.56	5.24	12 714.99	13 366.89	6.87
RC30-10-2	**14 812.04**	14 850.94	14 958.19	15 474.43	4.20	14 969.31	15 311.21	3.10
C30-10-3	9554.81	9752.94	**9491.78**	10 140.05	3.97	10 194.34	10 310.03	5.71
R30-10-3	**9975.68**	10 156.98	9987.64	10 315.27	1.56	10 725.19	11 012.47	8.42
RC30-10-3	**10 363.09**	10 670.53	10 413.15	10 884.61	2.01	11 004.33	12 038.41	12.82
C40-10-3	11 921.99	11 950.74	**11 673.39**	12 287.17	2.82	11 745.57	12 650.38	5.85
R40-10-3	11 985.00	12 020.17	11 871.57	12 393.93	3.11	**11 475.95**	12 718.28	5.81
RC40-10-3	**14 462.29**	14 580.89	15 860.99	16 388.22	12.40	14 581.96	15 478.29	6.15
C40-15-4	**9763.48**	9790.80	10 312.35	10 699.38	9.28	9908.38	10 370.47	5.92
R40-15-4	**10 303.94**	10 786.56	10 731.76	11 047.69	2.42	11 280.12	11 614.00	7.67
RC40-15-4	12 274.60	12 305.98	12 681.03	12 923.21	5.02	**12 242.48**	13 098.32	6.44
C50-15-3	12 630.14	12 652.45	**12 413.11**	12 651.41	−0.01	12 807.23	13 043.82	3.09
R50-15-3	16 272.24	16 853.52	**16 265.53**	17 049.71	1.16	17 321.73	17 581.57	4.32
RC50-15-3	18 137.26	18 400.52	**17 065.04**	18 160.93	−1.30	18 391.01	18 884.65	2.63
C50-15-4	**10 558.21**	10 714.15	10 918.43	11 203.57	4.57	11 127.84	11 407.51	6.47
R50-15-4	**13 063.67**	13 223.66	13 846.09	14 175.35	7.20	13 424.83	14 323.93	8.32
RC50-15-4	14 240.19	14 420.82	14 363.68	14 609.29	1.31	**14 202.42**	14 654.29	1.62
			Average Gap		**3.81**	**Average Gap**		**6.05**

The **bold** numbers represent the optimal values among these algorithms.

Table 11
Comparison with ALNS-VND and ALNS on large-sized instances.

Instance	MS-ALNS-VND		ALNS-VND			ALNS		
	C_{bst}	C_{avg}	C_{bst}	C_{avg}	Gap(%)	C_{bst}	C_{avg}	Gap(%)
C70-25-3	**17 776.21**	17 942.30	19 313.81	19 713.78	9.87	19 007.22	19 278.64	7.45
R70-25-3	**22 003.43**	22 070.31	23 113.99	24 008.70	8.78	22 252.94	23 319.48	5.66
RC70-25-3	**24 463.64**	24 679.51	27 068.10	27 629.79	11.95	25 511.51	26 300.93	6.57
C70-25-4	**14 948.89**	15 190.35	15 092.73	15 191.61	0.01	15 106.09	15 487.21	1.95
R70-25-4	**18 106.64**	18 203.80	18 430.31	18 586.90	2.10	19 524.22	19 630.03	7.83
RC70-25-4	**19 554.60**	20 103.62	19 816.75	20 542.41	2.18	21 557.94	21 664.15	7.76
C85-25-4	**19 872.60**	20 127.65	20 402.49	21 060.07	4.63	21 302.37	21 549.30	7.06
R85-25-4	**21 189.92**	21 575.53	22 269.46	22 736.37	5.38	22 503.03	23 073.06	6.94
RC85-25-4	**24 593.99**	25 203.74	25 760.08	26 127.54	3.67	26 015.53	26 763.09	6.19
C85-50-5	**17 595.59**	17 887.33	18 445.30	18 622.71	4.11	17 895.20	18 385.12	2.78
R85-50-5	**20 165.75**	20 845.84	22 122.91	22 348.38	7.21	22 308.25	22 653.97	8.67
RC85-50-5	23 330.70	23 464.01	**23 085.65**	23 450.16	−0.06	24 840.83	25 937.42	10.54
C100-25-5	**19 923.98**	20 251.56	20 365.72	20 897.77	3.19	21 078.92	21 378.11	5.56
R100-25-5	**20 779.10**	21 256.59	21 709.38	22 090.64	3.92	22 472.93	23 073.59	8.55
RC100-25-5	**22 908.05**	22 934.93	24 195.03	24 583.35	7.19	24 524.33	24 661.38	7.53
C100-50-5	**19 759.90**	20 154.96	20 437.64	20 979.92	4.09	21 444.79	21 932.36	8.82
R100-50-5	**23 813.57**	24 262.82	24 319.03	24 669.24	1.68	24 859.36	25 296.16	4.26
RC100-50-5	**27 480.38**	27 665.62	29 261.72	29 685.50	7.30	28 458.37	30 087.84	8.76
			Average Gap		**4.85**	**Average Gap**		**6.83**

The **bold** numbers represent the optimal values among these algorithms.

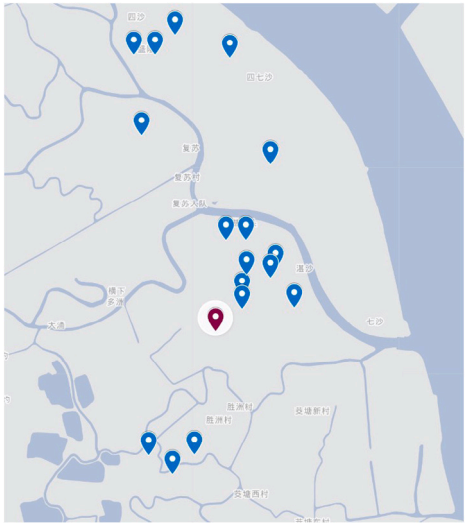


Fig. 8. The map of the electric automobile manufacturer and its suppliers.

Table 12
Impact of the number of vehicles on the objective of makespan.

K	C50-15-3	R50-15-3	RC50-15-3
3	12 595.00	17 029.89	18 121.61
5	10 191.36	12 815.48	13 059.73
7	10 031.86	10 917.53	11 021.93
9	10 002.86	10 320.00	10 767.67
11	10 007.55	10 301.00	10 700.91
13	9947.44	10 268.47	10 711.74

Table 13
The parameters in the real case.

Year	v	n	T	Γ	K	w	$C_{\max}$ (h)
2019	150	17	200	4320	6	4	7.47
2021	180	17	120	2590	9	4	7.32
2022	200	17	110	2370	10	4	7.21

Table 14
The optimized results of the case study.

Year	C_{max} (h)	C^*_{max} (h)	*Gap*(%)
2019	7.47	5.46	26.84
2021	7.32	5.91	23.76
2022	7.21	5.79	24.53

lower transportation costs. Secondly, reducing the number of vehicle trips helps decrease carbon emissions, aligning with the industry's environmental sustainability goals. Thirdly, the transportation of synchronized parts can be adjusted to accommodate various supplier and manufacturer schedules, offering flexibility in responding to changes in demand or supply. However, several challenges must be acknowledged to provide a comprehensive view of its industrial application. Firstly, the approach demands sophisticated coordination mechanisms among multiple suppliers, which can be challenging and resource-intensive to manage effectively. Secondly, the system's efficiency is highly contingent upon the punctuality and reliability of supplier deliveries, with potential disruptions affecting the entire logistics chain. Thirdly, there is often a need for an initial investment in technological and infrastructural upgrades, which could pose a financial barrier for some manufacturers considering the adoption of this system.

7. Conclusion

This paper investigated an integrated problem of car sequencing on an automotive mixed-model assembly line and multi-trip vehicle routing for external sync-part supply. The sync-parts, usually characterized by large size and highly customized, are directly delivered from suppliers to the assembly line without intermediate inventory. The goal is to find a joint schedule of car sequencing and vehicle routing so as to minimize the makespan.

To address the complex problem, a non-linear mixed-integer programming model is constructed and subsequently linearized into a mixed-integer linear programming model (MILP). Then we propose a new meta-heuristic (called MS-ALNS-VND) that incorporates multiple start strategy, adaptive large neighborhood search, and variable neighborhood descent. In addition, a novel encoding operator is devised to transform the delivery-assembly pull problem into an assembly-delivery push problem, effectively addressing the hard constraint imposed by the due dates of sync-parts. Moreover, we design swap, destroy, and local search operators that are customized to the problem's features to boost the algorithm's performance. To further avoid getting trapped in local optima, we reinforce the perturbation process with a simulated annealing strategy.

Numerical experimental results show that for small-sized instances, MS-ALNS-VND demonstrates remarkable performance by solving all 7 instances to optimality that Gurobi solver can do in significantly less running time. Moreover, it surpasses the Gurobi solver by rapidly obtaining superior solutions for an additional 10 instances where Gurobi fails to find optimal solutions. Whether in medium- or large-sized instances, MS-ALNS-VND finds more optimal solutions than ALNS-VND and ALNS do. In addition, MS-ALNS-VND achieves a better objective value than ALNS-VND and ALNS in most instances. In medium-sized instances, the average gaps of ALNS-VND and ALNS with our algorithm are 3.81% and 6.05%, respectively. In large-sized instances, the average gaps of ALNS-VND and ALNS with our algorithm are 4.85% and 6.83%, respectively.

A sensitivity analysis experiment shows that as the number of vehicles increases, there is a notable reduction in the average objective values, but as the number of vehicles continues to increase, the rate of reduction becomes less significant. This observation contributes to decision-making regarding vehicle procurement. Finally, a case study validates the applicability and effectiveness of the algorithm in practical problem-solving.

The research can be extended in several directions for future investigation. Firstly, incorporating a non-paced assembly line would introduce a more complex and challenging car sequencing problem. Secondly, our current study assumes fixed travel times, which do not fully reflect real-world conditions. Future work will account for more realistic factors such as dynamic travel times, variable vehicle numbers, and packing decisions. Thirdly, we plan to explore additional optimization objectives, such as minimizing average flow time, to capture different performance aspects. Finally, while our algorithm is tailored to ICSVRP, its applicability to other problems is limited. Reinforcement learning techniques may help generalize it to broader applications.

CRediT authorship contribution statement

Jian Chen: Writing – original draft, Supervision, Methodology, Funding acquisition, Formal analysis, Conceptualization. **Hong Zhou:** Writing – original draft, Validation, Methodology, Formal analysis. **Qiang Xue:** Validation, Software. **Naiming Xie:** Supervision, Resources, Investigation. **Yao-Wen Sang:** Writing – original draft, Supervision, Project administration, Methodology.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported by the National Natural Science Foundation of China (Grant Nos. 92367301, 52075259 and 72171116), the Qing Lan Project, China, and the China Postdoctoral Science Foundation (2021T140320 and 2019M661839).

Data availability

Data will be made available on request.

References

Avci, M. G., & Avci, M. (2019). An adaptive large neighborhood search approach for multiple traveling repairman problem with profits. *Computers & Operations Research*, *111*, 367–385.

Bautista, J., & Cano, A. (2011). Solving mixed model sequencing problem in assembly lines with serial workstations with work overload minimisation and interruption rules. *European Journal of Operational Research*, *210*(3), 495–513.

Belo-Filho, M., Amorim, P., & Almada-Lobo, B. (2015). An adaptive large neighbourhood search for the operational integrated production and distribution problem of perishable products. *International Journal of Production Research*, *53*(20), 6040–6058.

Blanchard, O. J. (1983). The production and inventory behavior of the American automobile industry. *Journal of Political Economy*, *91*(3), 365–400.

Boysen, N., & Emde, S. (2014). Scheduling the part supply of mixed-model assembly lines in line-integrated supermarkets. *European Journal of Operational Research*, *239*(3), 820–829.

Boysen, N., Fliedner, M., & Scholl, A. (2009). Sequencing mixed-model assembly lines: Survey, classification and model critique. *European Journal of Operational Research*, *192*(2), 349–373.

Calmels, D. (2022). An iterated local search procedure for the job sequencing and tool switching problem with non-identical parallel machines. *European Journal of Operational Research*, *297*(1), 66–85.

Cao, Z., Zhou, L., Lin, C., & Zhou, M. (2023). Solving an order batching, picker assignment, batch sequencing and picker routing problem via information integration. *Journal of Industrial Information Integration*, *31*, Article 100414.

Chang, Y.-C., Chang, K.-H., & Chang, T.-K. (2013). Applied column generation-based approach to solve supply chain scheduling problems. *International Journal of Production Research*, *51*(13), 4070–4086.

Chen, Z.-L. (2010). Integrated production and outbound distribution scheduling: review and extensions. *Operations Research*, *58*(1), 130–148.

Dayarian, I., & Desaulniers, G. (2019). A branch-price-and-cut algorithm for a production-routing problem with short-life-span products. *Transportation Science*, *53*(3), 829–849.

Defersha, F. M., & Mohebalizadehgashti, F. (2018). Simultaneous balancing, sequencing, and workstation planning for a mixed model manual assembly line using hybrid genetic algorithm. *Computers & Industrial Engineering*, *119*, 370–387.

Dhouib, S., Kharrat, A., & Chabchoub, H. (2010). A multi-start threshold accepting algorithm for multiple objective continuous optimization problems. *International Journal for Numerical Methods in Engineering*, *83*(11), 1498–1517.

Duhamel, C., Lacomme, P., Quilliot, A., & Toussaint, H. (2011). A multi-start evolutionary local search for the two-dimensional loading capacitated vehicle routing problem. *Computers & Operations Research*, *38*(3), 617–640.

Ehm, J., & Freitag, M. (2016). The benefit of integrating production and transport scheduling. *Procedia CIRP*, *41*, 585–590.

Elyasi, M., Selcuk, Y. S., Özener, O. Ö., & Coban, E. (2024). Imperialist competitive algorithm for unrelated parallel machine scheduling with sequence-and-machine-dependent setups and compatibility and workload constraints. *Computers & Industrial Engineering*, *190*, Article 110086.

Emde, S., & Boysen, N. (2012). Optimally routing and scheduling tow trains for JIT-supply of mixed-model assembly lines. *European Journal of Operational Research*, *217*(2), 287–299.

Facchini, F., Mossa, G., Sassanelli, C., & Digiesi, S. (2024). IoT-based milk-run routing for manufacturing system: an application case in an automotive company. *International Journal of Production Research*, *62*(1–2), 536–555.

Fattahi, P., & Salehi, M. (2009). Sequencing the mixed-model assembly line to minimize the total utility and idle costs with variable launching interval. *International Journal of Advanced Manufacturing Technology*, *45*, 987–998.

François, V., Arda, Y., & Crama, Y. (2019). Adaptive large neighborhood search for multitrip vehicle routing with time windows. *Transportation Science*, *53*(6), 1706–1730.

Golle, U., Rothlauf, F., & Boysen, N. (2014). Car sequencing versus mixed-model sequencing: A computational study. *European Journal of Operational Research*, *237*(1), 50–61.

Han, A. F.-W., & Chu, Y.-C. (2016). A multi-start heuristic approach for the split-delivery vehicle routing problem with minimum delivery amounts. *Transportation Research Part E: Logistics and Transportation Review*, *88*, 11–31.

Hashemi Doulabi, H., Pesant, G., & Rousseau, L.-M. (2020). Vehicle routing problems with synchronized visits and stochastic travel and service times: Applications in healthcare. *Transportation Science*, *54*(4), 1053–1072.

Hemmelmayr, V. C., Cordeau, J.-F., & Crainic, T. G. (2012). An adaptive large neighborhood search heuristic for two-echelon vehicle routing problems arising in city logistics. *Computers & Operations Research*, *39*(12), 3215–3228.

Karaoğlan, İ., & Kesen, S. E. (2017). The coordinated production and transportation scheduling problem with a time-sensitive product: A branch-and-cut algorithm. *International Journal of Production Research*, *55*(2), 536–557.

Kergosien, Y., Gendreau, M., & Billaut, J.-C. (2017). A Benders decomposition-based heuristic for a production and outbound distribution scheduling problem with strict delivery constraints. *European Journal of Operational Research*, *262*(1), 287–298.

Kuhn, H., Schubert, D., & Holzapfel, A. (2021). Integrated order batching and vehicle routing operations in grocery retail–a general adaptive large neighborhood search algorithm. *European Journal of Operational Research*, *294*(3), 1003–1021.

Lopes, T. C., Sikora, C. G. S., Michels, A. S., & Magatão, L. (2020). An iterative decomposition for asynchronous mixed-model assembly lines: Combining balancing, sequencing, and buffer allocation. *International Journal of Production Research*, *58*(2), 615–630.

Low, C., Chang, C.-M., & Gao, B.-Y. (2017). Integration of production scheduling and delivery in two echelon supply chain. *International Journal of Systems Science: Operations & Logistics*, *4*(2), 122–134.

Low, C., Li, R.-K., & Chang, C.-M. (2013). Integrated scheduling of production and delivery with time windows. *International Journal of Production Research*, *51*(3), 897–909.

Mao, Z., Huang, D., Fang, K., Wang, C., & Lu, D. (2020). Milk-run routing problem with progress-lane in the collection of automobile parts. *Annals of Operations Research*, *291*, 657–684.

Martí, R., Resende, M. G., & Ribeiro, C. C. (2013). Multi-start methods for combinatorial optimization. *European Journal of Operational Research*, *226*(1), 1–8.

Martins, L. d. C., Gonzalez-Neira, E. M., Hatami, S., Juan, A. A., & Montoya-Torres, J. R. (2021). Combining production and distribution in supply chains: The hybrid flow-shop vehicle routing problem. *Computers & Industrial Engineering*, *159*, Article 107486.

Nagano, M., Tomazella, C., Tavares-Neto, R., & Abreu, L. (2022). Solution methods for the integrated permutation flowshop and vehicle routing problem. *Journal of Project Management*, *7*(3), 155–166.

Nemoto, T., Hayashi, K., & Hashimoto, M. (2010). Milk-run logistics by Japanese automobile manufacturers in Thailand. *Procedia-Social and Behavioral Sciences*, *2*(3), 5980–5989.

Novaes, A. G., Bez, E. T., Burin, P. J., & Aragão Jr, D. P. (2015). Dynamic milk-run OEM operations in over-congested traffic conditions. *Computers & Industrial Engineering*, *88*, 326–340.

Özcan, U., Çerçioğlu, H., Gökçen, H., & Toklu, B. (2010). Balancing and sequencing of parallel mixed-model assembly lines. *International Journal of Production Research*, *48*(17), 5089–5113.

Pan, B., Zhang, Z., & Lim, A. (2021). Multi-trip time-dependent vehicle routing problem with time windows. *European Journal of Operational Research*, *291*(1), 218–231.

Parrello, B. D., Kabat, W. C., & Wos, L. (1986). Job-shop scheduling using automated reasoning: A case study of the car-sequencing problem. *Journal of Automated Reasoning*, *2*, 1–42.

Pontes, L., Neves, C., Subramanian, A., & Battarra, M. (2024). The maximum length car sequencing problem. *European Journal of Operational Research*, *316*(2), 707–717.

Potts, C. N. (1980). Analysis of a heuristic for one machine sequencing with release dates and delivery times. *Operations Research*, *28*(6), 1436–1441.

Ranjbaran, F., Husseinzadeh Kashan, A., & Kazemi, A. (2020). Mathematical formulation and heuristic algorithms for optimisation of auto-part milk-run logistics network considering forward and reverse flow of pallets. *International Journal of Production Research*, *58*(6), 1741–1775.

Ropke, S., & Pisinger, D. (2006). An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows. *Transportation Science*, *40*(4), 455–472.

Rutenbar, R. A. (1989). Simulated annealing algorithms: An overview. *IEEE Circuits and Devices Magazine*, *5*(1), 19–26.

Sadjadi, S., Jafari, M., & Amini, T. (2009). A new mathematical modeling and a genetic algorithm search for milk run problem (an auto industry supply chain case study). *International Journal of Advanced Manufacturing Technology*, *44*, 194–200.

Schmid, N. A., Limère, V., & Raa, B. (2021). Mixed model assembly line feeding with discrete location assignments and variable station space. *Omega*, *102*, Article 102286.

Shaw, P. (1997). A new local search algorithm providing high quality solutions to vehicle routing problems. *APES Group, Dept of Computer Science, University of Strathclyde, Glasgow, Scotland, UK*, *46*.

Solomon, M. M. (1987). Algorithms for the vehicle routing and scheduling problems with time window constraints. *Operations Research*, *35*(2), 254–265.

Sun, H. (2024). Integrating virtual resequencing with car resequencing via selectivity banks for mixed-model assembly lines. *Computers & Industrial Engineering*, *189*, Article 109990.

Tamura, T., Okumura, T., Singh Dhakar, T., & Ohno, K. (2011). Optimal production sequencing problem to minimise line stoppage time in a mixed-model assembly line. *International Journal of Production Research*, *49*(14), 4299–4315.

Tasoglu, G., & Ilgin, M. A. (2024). A simulation-based genetic algorithm approach for the simultaneous consideration of reverse logistics network design and disassembly line balancing with sequencing. *Computers & Industrial Engineering*, *187*, Article 109794.

Tiacci, L. (2024). Combining balancing, sequencing and buffer allocation decisions to improve the efficiency of mixed-model asynchronous assembly lines. *Computers & Industrial Engineering*, *194*, Article 110357.

Ullrich, C. A. (2013). Integrated machine scheduling and vehicle routing with time windows. *European Journal of Operational Research*, *227*(1), 152–165.

Villegas, J. G., Prins, C., Prodhon, C., Medaglia, A. L., & Velasco, N. (2010). GRASP/VND and multi-start evolutionary local search for the single truck and trailer routing problem with satellite depots. *Engineering Applications of Artificial Intelligence*, *23*(5), 780–794.

Wang, H., Sheng, B., Lu, X., Fu, G., & Luo, R. (2022). Task package division method for the integrated scheduling framework of mixed model car-sequencing problem. *Computers & Industrial Engineering*, *169*, Article 108144.

Zhen, L., Ma, C., Wang, K., Xiao, L., & Zhang, W. (2020). Multi-depot multi-trip vehicle routing problem with time windows and release dates. *Transportation Research Part E: Logistics and Transportation Review*, *135*, Article 101866.

Zhou, B., & Zhu, Z. (2020). Optimally scheduling and loading tow trains of in-plant milk-run delivery for mixed-model assembly lines. *Assembly Automation*, *40*(3), 511–530.